Lab1: Back-propagation Report

311351009 劉承霈

1. Introduction

In this lab, I implemented some simple neural networks with two hidden layers, focusing on the forward pass and updating weights via back-propagation to minimize the loss. In addition to the basic setup, I also tried using different learning rates, numbers of hidden units, activation functions, etc. to compare the difference in loss between them.

2. Experiment setups

A. Sigmoid functions

- I use the Sigmoid function as the default activation function.
- The followings are the implementation of the Sigmoid function and its derivative:

```
def activation(x, act):
    if act == "Sigmoid":
        return 1.0 / (1.0 + np.exp(-x))

def d_activation(x, act):
    if act == "Sigmoid":
        return np.multiply(x, 1.0 - x)
```

B. Neural network

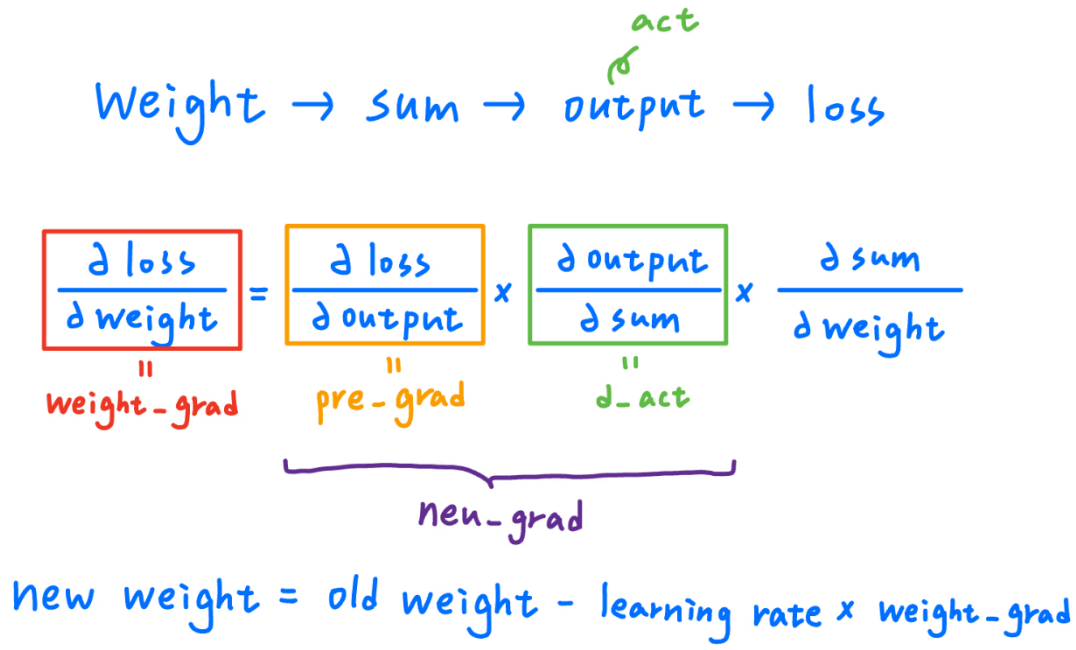
- I create two classes **Layer** and **NeuralNetwork** to build my neural network.
  - class **Layer** focuses on one single layer implementing forward, backward, and update functions.
  - class **NeuralNetwork** first constructs a neural network, and then implements multiple layers of forward, backward and update functions.
- My neural network consists of 1 input layer containing 2 neurons, 2 hidden layers containing 4 neurons, and 1 output layer containing 1 neuron.
- The followings are the details of each function:

- forward(): Multiply the input data with the weights, and pass it to the activation function to generate the output value
- backward(): Apply Chain rule to iteratively compute gradients for each layer. (More details in C. Backpropagation)
- update(): Use the gradient of the backward result to perform gradient descent to update the weights.

C. Backpropagation

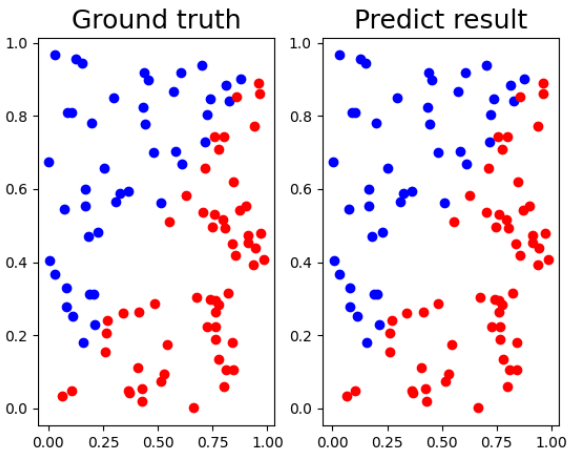
- The aim of backpropagation is to distribute the total loss back to the network so as to update the weights in order to minimize the loss.
- If we look closely at the neural network, we can see that:
  - "Loss" is affected by total "output"
  - "Output" is affected by "sum" of weights
  - "Sum" is affected by "weight"

  Therefore, we can apply Chain Rule to calculate the weight_gradient and update the weights: (The notations correspond to my code.)

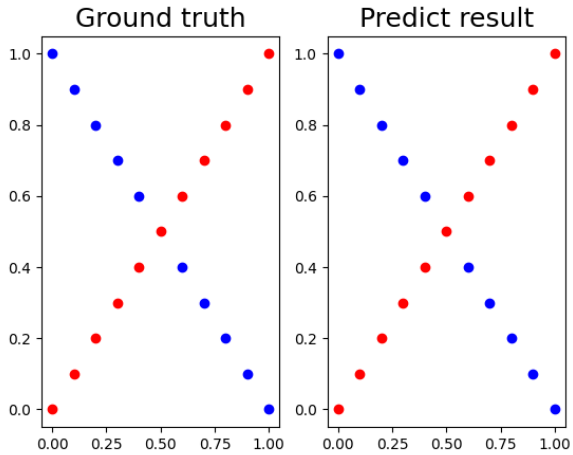


3. Results of your testing

A. Screenshot and comparison figure



(a) Linear



(b) XOR

B. Show the accuracy of your prediction

· Linear:

```
Iter91  |     Ground truth: 1.0  |     prediction: 0.99961  |     accuracy: 0.92000000  |
Iter92  |     Ground truth: 0.0  |     prediction: 0.00058  |     accuracy: 0.93000000  |
Iter93  |     Ground truth: 1.0  |     prediction: 0.87675  |     accuracy: 0.94000000  |
Iter94  |     Ground truth: 0.0  |     prediction: 0.00585  |     accuracy: 0.95000000  |
Iter95  |     Ground truth: 0.0  |     prediction: 0.00103  |     accuracy: 0.96000000  |
Iter96  |     Ground truth: 1.0  |     prediction: 0.99964  |     accuracy: 0.97000000  |
Iter97  |     Ground truth: 1.0  |     prediction: 0.82663  |     accuracy: 0.98000000  |
Iter98  |     Ground truth: 0.0  |     prediction: 0.00059  |     accuracy: 0.99000000  |
Iter99  |     Ground truth: 1.0  |     prediction: 0.99523  |     accuracy: 1.00000000  |
loss=0.01043 accuracy=100.0%
```

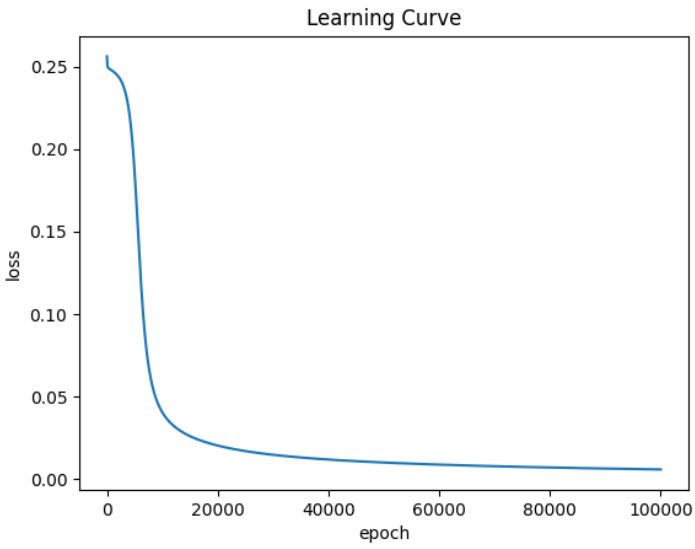
- XOR:

```
Iter 0  |     Ground truth: 0.0  |     prediction: 0.01293  |     accuracy: 0.04761905  |
Iter 1  |     Ground truth: 1.0  |     prediction: 0.99996  |     accuracy: 0.09523810  |
Iter 2  |     Ground truth: 0.0  |     prediction: 0.01297  |     accuracy: 0.14285714  |
Iter 3  |     Ground truth: 1.0  |     prediction: 0.99996  |     accuracy: 0.19047619  |
Iter 4  |     Ground truth: 0.0  |     prediction: 0.01301  |     accuracy: 0.23809524  |
Iter 5  |     Ground truth: 1.0  |     prediction: 0.99994  |     accuracy: 0.28571429  |
Iter 6  |     Ground truth: 0.0  |     prediction: 0.01304  |     accuracy: 0.33333333  |
Iter 7  |     Ground truth: 1.0  |     prediction: 0.99986  |     accuracy: 0.38095238  |
Iter 8  |     Ground truth: 0.0  |     prediction: 0.01306  |     accuracy: 0.42857143  |
Iter 9  |     Ground truth: 1.0  |     prediction: 0.96732  |     accuracy: 0.47619048  |
Iter10  |     Ground truth: 0.0  |     prediction: 0.01306  |     accuracy: 0.52380952  |
Iter11  |     Ground truth: 0.0  |     prediction: 0.01306  |     accuracy: 0.57142857  |
Iter12  |     Ground truth: 1.0  |     prediction: 0.97125  |     accuracy: 0.61904762  |
Iter13  |     Ground truth: 0.0  |     prediction: 0.01305  |     accuracy: 0.66666667  |
Iter14  |     Ground truth: 1.0  |     prediction: 0.99987  |     accuracy: 0.71428571  |
Iter15  |     Ground truth: 0.0  |     prediction: 0.01304  |     accuracy: 0.76190476  |
Iter16  |     Ground truth: 1.0  |     prediction: 0.99992  |     accuracy: 0.80952381  |
Iter17  |     Ground truth: 0.0  |     prediction: 0.01301  |     accuracy: 0.85714286  |
Iter18  |     Ground truth: 1.0  |     prediction: 0.99992  |     accuracy: 0.90476190  |
Iter19  |     Ground truth: 0.0  |     prediction: 0.01298  |     accuracy: 0.95238095  |
Iter20  |     Ground truth: 1.0  |     prediction: 0.99991  |     accuracy: 1.00000000  |
loss=0.00018 accuracy=100.0%
```

C. Learning curve (loss, epoch curve)

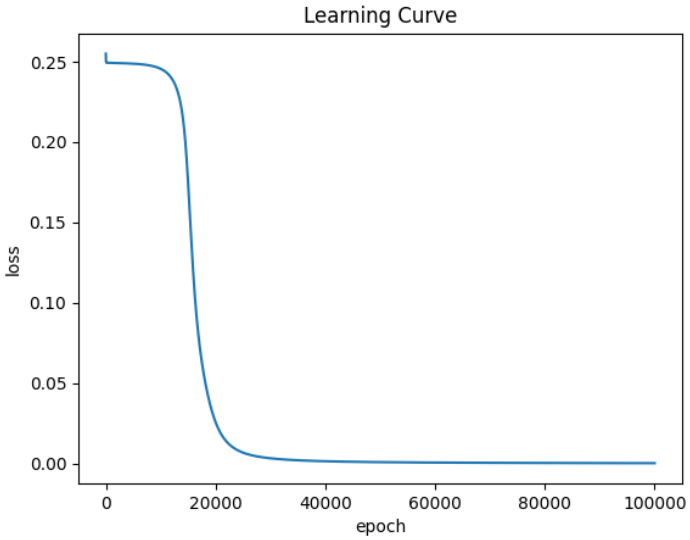
- Linear:

```
epoch 10000 loss : 0.0405991600246411
epoch 15000 loss : 0.0261750699552248
epoch 20000 loss : 0.0204784669458921
epoch 25000 loss : 0.0171385940414918
epoch 30000 loss : 0.0148908220487886
epoch 35000 loss : 0.0132596222833541
epoch 40000 loss : 0.0120124386358814
epoch 45000 loss : 0.0110204019472372
epoch 50000 loss : 0.0102064975006315
epoch 55000 loss : 0.0095220585845617
epoch 60000 loss : 0.0089348997153033
epoch 65000 loss : 0.0084229005340685
epoch 70000 loss : 0.0079703514162618
epoch 75000 loss : 0.0075657785171069
epoch 80000 loss : 0.0072005979941875
epoch 85000 loss : 0.0068682538382408
epoch 90000 loss : 0.0065636481540366
epoch 95000 loss : 0.0062827542056403
```



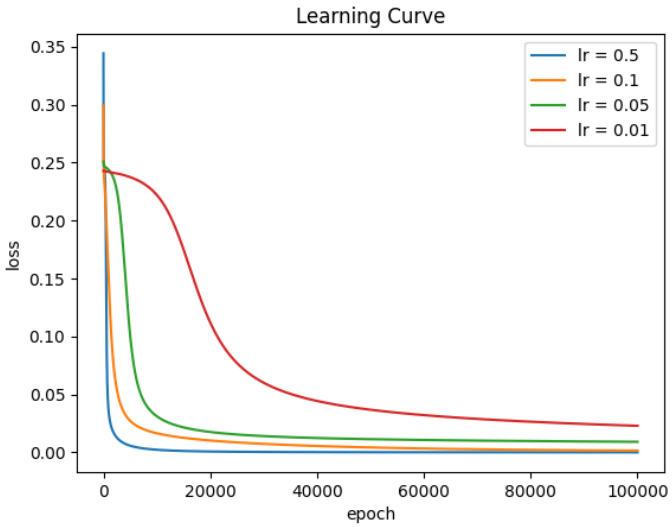
- XOR:

```
epoch 10000 loss : 0.2455952700580447
epoch 15000 loss : 0.1743229692537678
epoch 20000 loss : 0.0258073463230728
epoch 25000 loss : 0.0068658374960355
epoch 30000 loss : 0.0033424629835235
epoch 35000 loss : 0.0021092725633092
epoch 40000 loss : 0.0015124473242585
epoch 45000 loss : 0.0011678895921748
epoch 50000 loss : 0.0009460482329484
epoch 55000 loss : 0.0007922961457886
epoch 60000 loss : 0.0006799401645816
epoch 65000 loss : 0.0005944993951222
epoch 70000 loss : 0.0005274807111222
epoch 75000 loss : 0.0004735925132384
epoch 80000 loss : 0.0004293750849987
epoch 85000 loss : 0.0003924756563554
epoch 90000 loss : 0.0003612410241673
epoch 95000 loss : 0.0003344770004185
```

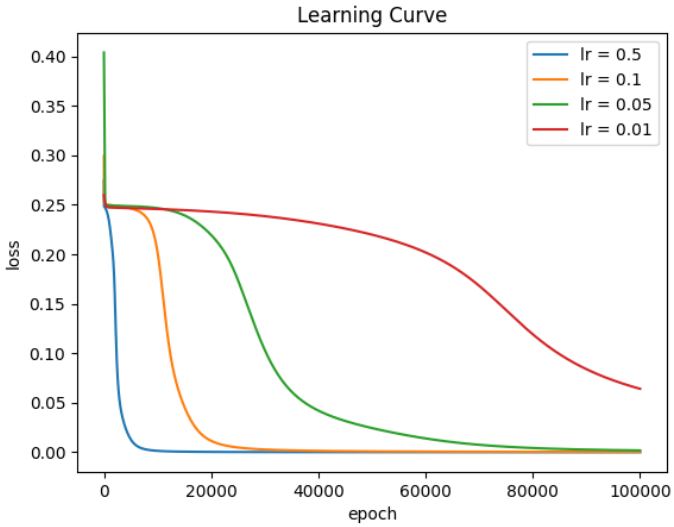


4. Discussion

A. Try different learning rates



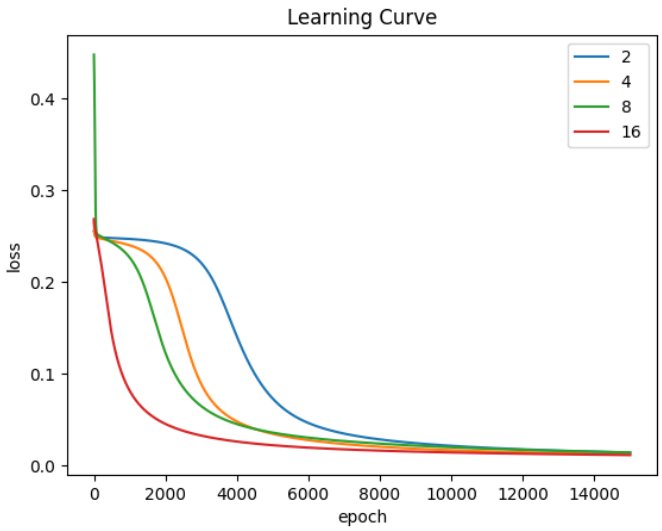
(a) Linear



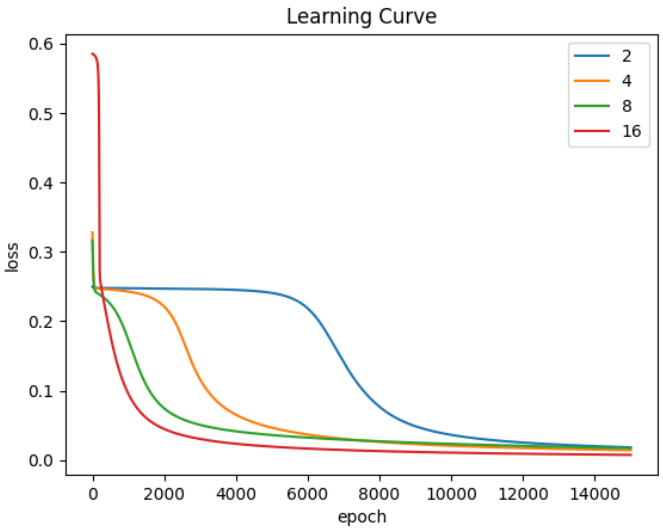
(b) XOR

- In both cases, higher learning rate results in faster convergence, leading to a quicker decrease in the loss function. This is because larger steps are taken during the weight updates, allowing the model to reach the optimal solution more rapidly.
- However, I think in other cases, using an excessively high learning rate may cause negative consequences, such as oscillation in loss function or failure to converge altogether.

B. Try different numbers of hidden units



(a) Linear

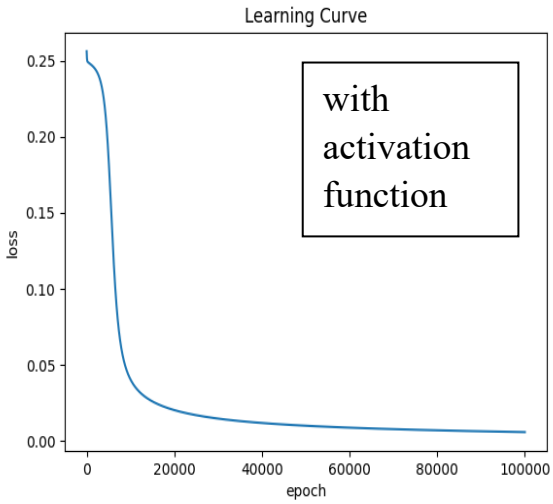


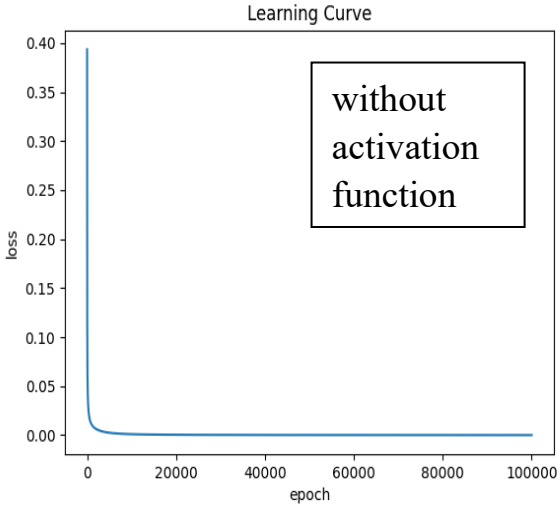
(b) XOR

- In both cases, increasing the number of hidden units helps improve the model's performance and reduce the loss. This is because additional hidden units provide the neural network with more capacity to learn complex patterns.
- However, I think in other cases, adding more hidden units may lead to overfitting or increasing the model's complexity and computation.

C. Try without activation functions

- Linear:

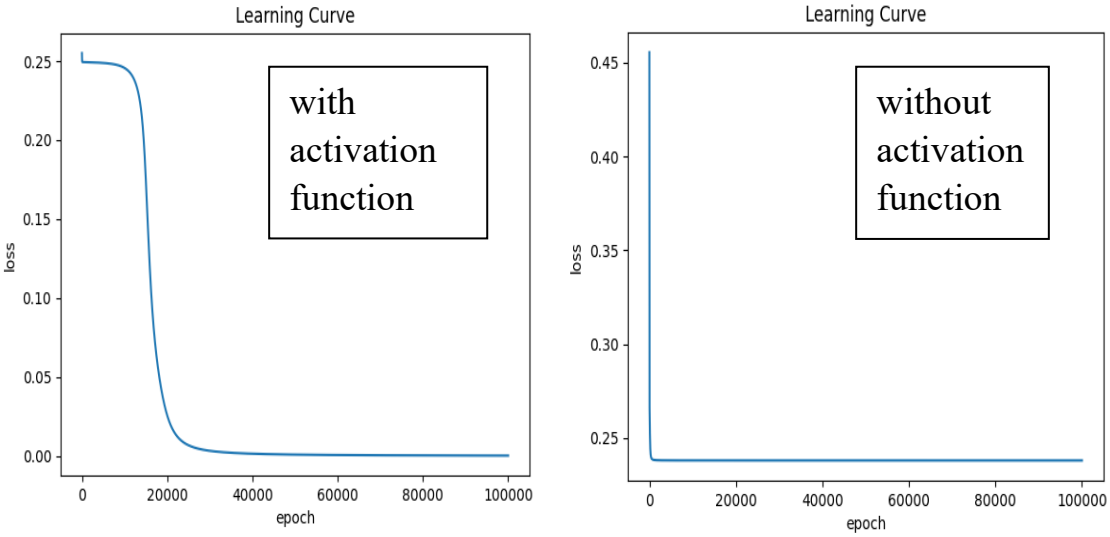




Since the activation function is used to convert linear to nonlinear, in this linear case, the losses with/without activation function appear to converge

in the similar range, but convergence would be faster without activation function.

- XOR:

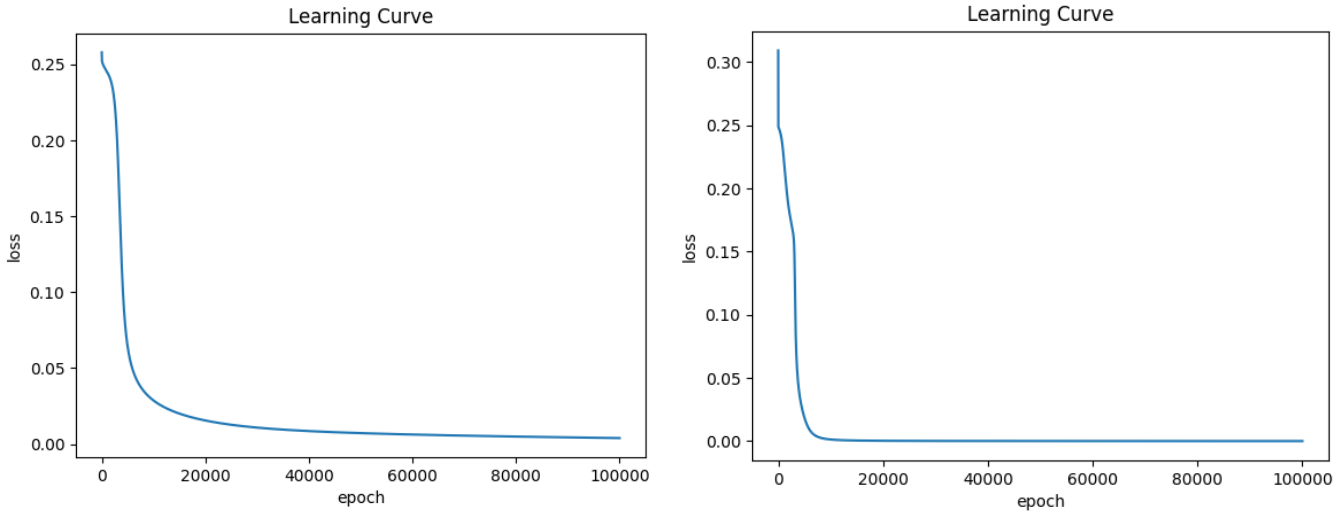


For this nonlinear case, loss with activation function is much lower than the one without activation function.

5. Extra

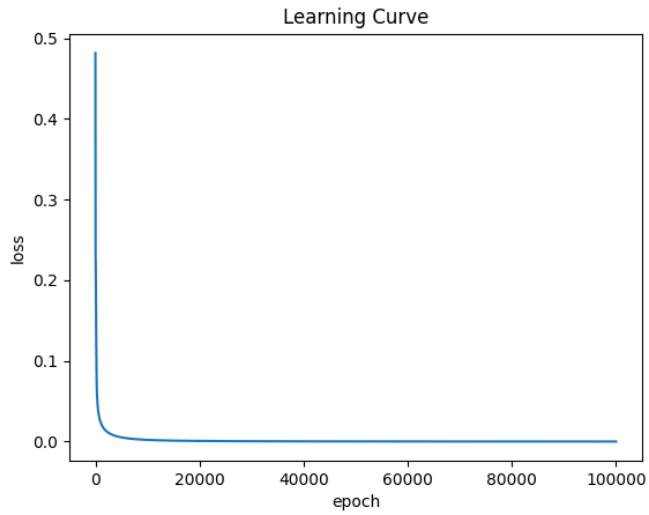
B. Implement different activation functions.

i. Sigmoid:

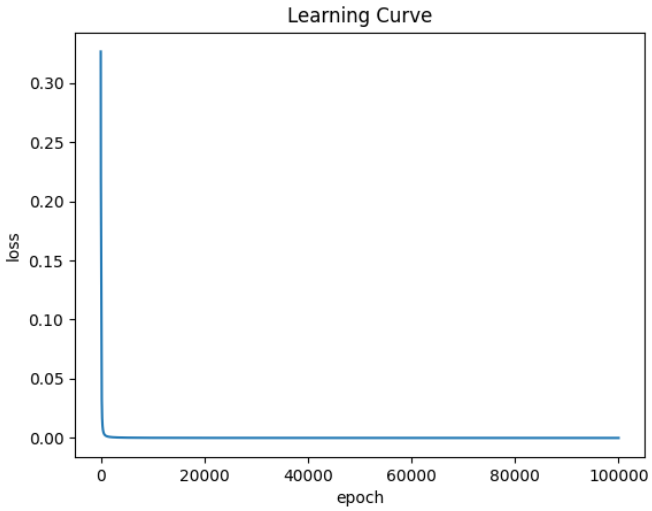


(a) Linear (b) XOR

ii. tanh:

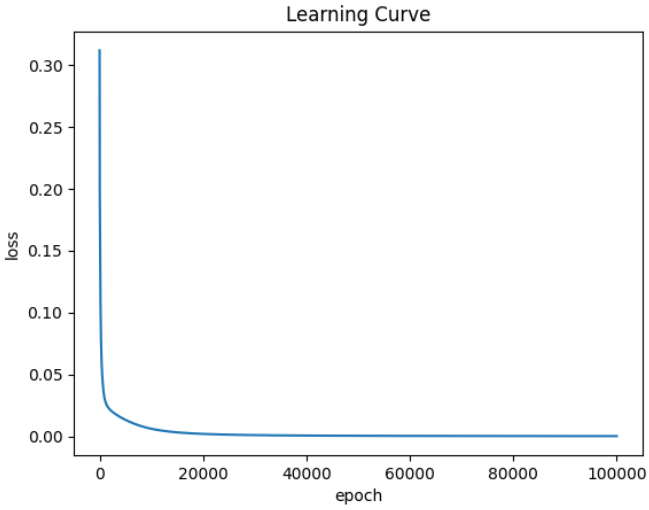


(a) Linear

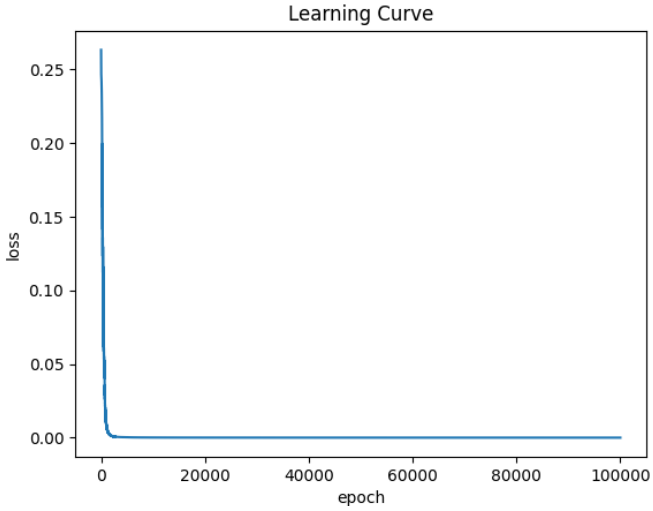


(b) XOR

iii. ReLU:



(a) Linear



(b) XOR